

Tools & Plugin API

Blender Tool, Canvas, Slidev, Skills, Bot Commands

March 31, 2026

ClawCraft Project Reference

Plugin: blender (core tool)

Registered in extensions/blender/src/blender-tool.ts. Controls Blender via TCP socket to the MCP addon on port 9876.

Actions

- **scene_info** Query all objects, materials, and scene metadata. No parameters required. Returns JSON.
- **object_info** Get detailed info for a named object. Requires: name (string).
- **execute** Run arbitrary Blender Python code. bpy is pre-imported. 180-second timeout. Requires: code (string).
This is how all scenes are built -- the LLM writes Python that creates meshes, assigns materials, places cameras, sets up lights.
- **render** Render the current scene. Forces EEVEE at 1920x1080 RGBA PNG. Auto-increments iteration per scene_id. Auto-logs to render_log.json. Auto-pushes to Notion Renders DB. Auto-updates Scene row. Returns image via MEDIA: directive. Requires: scene_id, user_prompt. Optional: refinement_notes, outcome_type, workflow_observation, engine, width, height, samples.
- **log_note** Append a note to an existing render entry. Requires: render_id, note.
- **rate** Set quality rating 1-5 on a render entry. Requires: render_id, rating.

Render Parameters

- **scene_id (string)** Scene identifier for logging (e.g., fridge_portal). Required.
- **user_prompt (string)** The user's original request. Required.
- **refinement_notes (string)** Why this iteration exists, what changed. Optional.
- **outcome_type (string)** success|partial|failure|happy_accident. Auto-set, overridable. Optional.
- **workflow_observation (string)** One sentence about the PROCESS, not the image. Optional.
- **engine (string)** BLENDER_EEVEE (default) or BLENDER_EEVEE_NEXT. Never Cycles.
- **width (number)** Render width in pixels (default: 1920)
- **height (number)** Render height in pixels (default: 1080)

Render Flow (internal)

1. Generate Python code: set engine, resolution, render to temp file, base64-encode result
2. Send code via TCP socket to Blender addon, 300s timeout
3. Parse base64 image from Blender stdout marker
4. Save PNG to /home/node/.openclaw/media/renderers/{scene_id}_{iteration:003}.png
5. Create RenderLogEntry, append to render_log.json
6. Fire pushRenderToNotion() (async, best-effort, non-blocking)
7. Return MEDIA:{filepath} in response text

Notion Integration (in render)

pushRenderToNotion() uses Node fetch() to call Notion REST API v2022-06-28:

1. POST /databases/{SCENES_DB}/query to find scene page ID by name
2. POST /pages to create Renders row with all fields + Scene relation
3. PATCH /pages/{scenePageId} to update Iterations count + Status

Plugin: canvas

HTML-to-image renderer. Pass any HTML content, get a PNG screenshot via headless Chromium (Playwright). Images saved to media/renders/ and returned via MEDIA: directive.

CDN Libraries (zero install, loaded via script tag)

- **Mermaid** Flowcharts, sequence diagrams, Gantt charts, ERD, class diagrams
 - **KaTeX** LaTeX math rendering
 - **Shiki** Syntax-highlighted code blocks (100+ languages)
 - **Iconify** 10,000+ icons from multiple icon sets
 - **D3.js** Advanced data visualization, custom SVG charts
 - **Chart.js** Simple bar, line, pie, doughnut, radar charts
-

Plugin: slidev

Presentation generator. Bot writes Markdown, Slidev builds into static HTML deck, Playwright screenshots each slide, all posted to Discord as MEDIA: images.

- Theme: slidev-theme-light-icons
 - Addon: slidev-addon-python-runner (execute Python in slides)
 - Addon: plotly.js-dist-min (interactive-quality charts in slides)
 - Use case: generating the final presentation from render_log.json data
-

Skills (10 enabled)

- **discord** Send messages, media, reactions, create threads, read/search messages, set presence
 - **openai-image-gen** DALL-E / gpt-image-1 concept art. COSTS MONEY per image (\$0.04-0.08). Required as second ML architecture for rubric.
 - **github** Full gh CLI access. Create repos, issues, PRs, manage releases.
 - **nano-pdf + pypdf** PDF editing with natural language + Python PDF library
 - **video-frames** Extract frames from video via ffmpeg. Reference material, animation frames.
 - **weather** Weather lookups (from prior OpenClaw demo)
 - **healthcheck** System health verification
 - **skill-creator** Create new skills at runtime
 - **asciinema** Terminal session recording for demo videos
-

Python Packages (in container)

- **matplotlib + pillow** Charts, graphs, image manipulation
- **plotly** Rich interactive-quality charts, 3D plots, heatmaps

- **yt-dlp** Video downloads (reference material)
 - **nano-pdf + pypdf** PDF editing and reading
-

Bot Commands

- **/log-note render_001 "text"** Append a note to render log entry
- **/rate render_001 4** Set quality rating 1-5 on a render
- **/session-summary** Post today's render stats, outcome breakdown, top workflow observation
- **/render-report** Generate full Slidev presentation from render_log.json data
- **"run next prompt"** Start the next unstarted scene from prompt_queue.md